

Advanced Software Development for Robotics

Assignment manual (v2025.1)

Jan Broenink, Gijs van Oort, Islam Khalil

Monday 26th January, 2026

Contents

0	Introduction	1
0.1	Context	1
0.2	Organisation	1
0.3	Support from the Teaching Assistants	2
0.4	Responsibility and fraud	2
0.5	Document History and Disclaimer	3
1	Assignment 1 — ROS2	4
1.0	Assignment 1.0: Getting familiar with ROS2	4
1.1	Assignment 1.1: Image processing with ROS2	4
1.2	Assignment 1.2: Sequence controller	5
1.3	Note – for both PPD and Classical versions of the course	8
2	Assignment 2.0: Timing in Robotic / Cyber-Physical Systems	9
2.0	Introduction	9
2.1	Assignment 2.1: Timing of a periodic non-real-time thread (Ubuntu)	10
2.2	Assignment 2.2: Timing of periodic real-time thread (Xenomai)	11
2.3	<i>Classical only:</i> Assignment 2.3: Timing of two communicating ROS2 nodes	12
2.4	Assignment 2.4: RELbot CPS Architecture Diagrams	12
3	Assignment 3 — Integration, Controlling a robot	15
3.0	Introduction	15
3.1	Assignment 3.1: Controller-code stack on the Raspberry Pi of the RELbot	16
3.2	Assignment 3.2: XRF2: the FRT software architecture on RELbot's Raspberry Pi	17
3.3	Assignment 3.3: Loop-Controller algorithm in FRT part	18
3.4	Assignment 3.4: Final integration	19
3.5	Assignment 3.5: Extra functionality (Bonus assignment)	20
3.6	Final Submission of Assignment 3	21
A	Changelog	22
B	Course-specific framing for PPD projects	23
B.1	Describe the CPS architecture of your robotic system – both SDfR and ASDfR	23
B.2	Specifics for ASDfR	23
B.3	Combined SDfR and ASDfR	24
B.4	Hints on producing diagrams	24
	Bibliography	25

0 Introduction

0.1 Context

The goal of this practical work is get familiar with soft real-time and hard real-time software development, and the combination of it, in the context of controlling robotic/mechatronic systems (physical machines). We use the RELbot, the in-house developed education mobile robot of the Robotics Education Lab of MSc Robotics.

For the soft real-time part (Assignment set 1 and 3), we use ROS2 (Robotic Operating System 2), version Jazzy, heavily used in modern robotics. The algorithms are programmed in C++. For the development environment, we use a mainstream development tool, VS Code, and run ROS in a dedicated environment, using the virtualisation tool WSL (on Windows) or Multipass (on Mac). ROS needs a particular version of Linux to run in (Ubuntu 2024.04), and that is provided via the virtualisation tools.

Next to this, migrating to developing for the RELbot is straightforward, as the embedded computer is just another dedicated environment, connected to the host machine via a normal network.

For installing necessary tools on your laptop, software development for RELbots, and use of RELbots, see the *RELbot Software and Usage Manual (v2025.n)* by van Oort et al. (2025), in this assignment manual referred to as the *RELbot Manual*.

For the hard real-time part (Assignment sets 2 and 3), we work with a Raspberry Pi4, using a specifically configured Linux (Ubuntu with the Xenomai real-time patch) and programs written in C++. The Raspberry Pi4s are available at the RaM Lab as control computers of the RELbots. In Assignment set 2, only the Raspberry Pi4 is used on its own, so without the add-on boards and rest of the drive train. In Assignment set 3, the RELbot is controlled using its Raspberry Pi.

The focus in the assignments of this course lies on developing *good, well-structured solutions*. Code that is ‘hacked together’ and lacks good structure or programming style does *not* receive full points, even if the output of the program is perfect.

0.2 Organisation

The practical assignments consist of three assignment sets, and must be done in groups of 2; all three assignment sets with the same team of 2 students. Results must be submitted via the UT Canvas page of the course (other ways of submission are *not* accepted). You must supply a report (as PDF) as well as a structured zip file with *all* source code (separated per sub-assignment). The code should be such that it can be compiled (and run) by the teaching assistants on their own computer. Deadlines are published on Canvas.

0.2.1 Requirements for Assignment Reports

General requirements for the reports are:

- Reports must be compact: only answers need to be given, but always provide a motivation.
- Use a readable, *one-column* layout.
- Include screen dumps of models / diagrams and results in the report.
- For ROS2 programs, *always* put a node/topic graph in the report (`rqt_graph`).
- For ROS2 programs, submit the structure as indicated in Chapter 6 of the RELbot manual (van Oort et al., 2025).
- Put also relevant code snippets in the report, but never put the full code in the report, not even as an appendix.

- The sentence *Show that the system works* as regularly written as part of an assignment task, means that you set up, run, and report on some tests to make clear that the system does what you specify it should do. You should address and discuss any anomalies.

Note that the teacher only reads the report and the teaching assistants also check the code in the zip file.

0.2.2 PPD version versus Classical version of the course

MSc-Robotics students who take this class as compulsory course must do the PPD¹ variant of this course. This means, they can skip certain sub-assignments (those are indicated), and contribute to their PPD case using material from this course. Next to the *framing* Section of the PPD manual, more information is in Appendix B.

Those who do *not* need to do the PPD version, do the *Classical* version of this course, that is all the assignments in this document.

0.3 Support from the Teaching Assistants

There is a weekly Q&A directly after the lecture, so Wednesdays or Tuesdays, 12:45-13:30, in CR 3446, the large meeting room of RaM, directly starting in the first week. You could use the first occasion for support on installing the tools.

During the Lab sessions of Assignment Set 2 and 3, teaching assistants are of course present in the Lab.

You can ask questions regarding the assignments (most notably Assignment Set 1) via *Discussions* available on the Canvas page.

At times when teaching assistants are present (Wednesday's Q&A, Lab sessions) we run the queueing tool Horus to handle local and online questions synchronously.

0.4 Responsibility and fraud

All group members share responsibility for the work handed in. When you distribute work among group members, check each other's work, discuss the work among all group members, do you understand it all?

You must submit original work.

Plagiarism, i.e. copying of someone else's work without proper reference, is a serious offence which in all cases will be reported to the Examination Board. Refer to UT's Student Charter Student Charter (2020).

In cases where a non-trivial amount of work is copied *with* proper reference, indicate which parts are copied and which parts are your own original work. The copied work will not be considered for grading.

0.4.1 What constitutes fraud?

Material written by Arend Rensink is reused here.

When it comes down to handing in assignments, every year there are students who do not understand the borderline between, on the one hand, cooperating and discussing solutions between groups (which is allowed), and on the other, copying or sharing solutions (which is forbidden and counted as fraudulent behaviour). Here are some scenarios which may help in making this distinction.

- **Scenario.** Peter and Lisa are quite comfortable with programming and have pretty much finished the assignment. Mark and Wouter, on the other hand, are struggling and ask Lisa

¹Professional and Personal Development, formerly named Challenge-Based Learning.

how she has solved it. Lisa, a friendly girl, shows her solution and takes them through it line by line. Mark and Wouter think they now understand and go off to create their own solution, based on what they saw. Is this allowed or not?

Verdict. No problem here, everything is in the green. It is perfectly fine and allowed for Lisa to explain her solution, even very thoroughly. The important point is that in implementing it themselves and testing their own solution, Mark and Wouter are still forced to think about what is happening and will gain the required understanding, though probably they will not get as much out of it as Lisa (explaining stuff to others is about the best possible way to learn it better yourself!).

- **Scenario.** The start is as in the previous case. However, while Mark and Wouter implement their own solution, inspired by that of Lisa, some error crops up which they do not understand. Lisa has left by now; after they mail her, still trying to be helpful she sends them her solution for them to inspect. They inspect it so closely that in the end their solution is indistinguishable from Peter and Lisa's, except for the choice of some variable names and the comments they added themselves. Is this allowed or not?

Verdict. This is now a case of fraud. Three are at fault: Lisa for enabling fraud by sending her files (even if it was meant as a friendly gesture) and Mark and Wouter for copying the code. Peter was not involved, developed his own solution (together with Lisa) and is innocent.

- **Scenario.** Alexandra and Nahuel are not finished, and the deadline is very close. The same holds for Simon and Jaco. On the Friday night train home, Jaco and Nahuel meet and during the 2-hour train ride work it out together. They type in the same solution and hand it in on behalf of their groups. Is this allowed or not?

Verdict. This is also a case of fraud. Actually there are two problems here. The first is that both Nahuel and Jaco handed in code on behalf of their groups that had been developed by them alone, without their partners. This is unwise and against the spirit of the assignment (Alexandra and Simon also need to master this stuff!) but essentially undetectable and not fraudulent. The second problem is that the solution was developed, and shared, in collaboration between two groups; this is definitely forbidden. All four students are culpable; Alexandra and Simon cannot hide behind the fact that they did not partake in the collaboration, as they were apparently happy enough to have their name on the solutions and pretend they worked on it, too.

Note that we are not on a witch-hunt here: let us stress again that cooperating and discussing assignments is OK, even encouraged; it is at the point where you start copying or duplicating pieces of code that you cross the border.

0.5 Document History and Disclaimer

When you find a mistake or have remarks about this document, please contact one of the TAs or the teacher.

As we also continuously improve this document, newer versions appear regularly, see its date and version number. Changes are summarized in the Changelog, in Appendix A. The latest version is on Canvas. Be sure to always use the latest version of this guide.

This document has seen quite some versions over the years. Contributors to this document, next to the teacher, are: Jan Broenink, Gijs van Oort, Luuk Lenders, Ilyas Raoudi, Jasper Vinken-vleugel, Jurriijn Pott, Frank Hooglander, Arnold Hofstede, and Nick in 't Veld.

Development and documentation of the RELbot is by Gijs van Oort, Sander Smits, Marcel Schwirtz, Marjon Kuipers, Artur Gășienica, and Jan Broenink.

Special thanks to Arend Rensink for the Subsection "What constitutes fraud?".

1 Assignment 1 — ROS2

In this assignment you get familiar with ROS2 and build some projects with it. See the *RELbot Software and Usage Manual (v2025.n)*, van Oort et al. (2025) for how to get access to ROS2 and install the required software.

1.0 Assignment 1.0: Getting familiar with ROS2

ROS2 has good tutorials on the website:

<https://docs.ros.org/en/humble/Tutorials.html>

Do (at least) the following tutorials:

1. Using turtlesim and rqt
2. Understanding nodes
3. Understanding topics
4. Launching nodes
5. Creating a workspace
6. Creating a package
7. Writing a simple publisher and subscriber (C++)
8. Using parameters in a class (C++)

You do not have to hand in anything for this assignment.

1.1 Assignment 1.1: Image processing with ROS2

To get more familiar with ROS2, we do some simple image processing work. However, the focus of this assignment is *not* the image processing itself, but rather the structure of the nodes and topics in ROS2.

1.1.1 Camera input with standard ROS2 tools

Due to Multipass VM or WSL Linux *not* being able to access the webcam directly, the webcam stream must be sent over a UDP channel. For this, we provide a videosever script and an updated image_tools ROS2 package, see Chapter 7 of the *RELbot Software and Usage Manual (v2025.n)*, van Oort et al. (2025). It is available on Canvas.

Use the specific image_tools ROS2 package (from Canvas), and the standard showimage node of the standard image_tools ROS2 package to capture webcam footage, publish to a channel and view the channel.

The standard showimage node can be started using the following command in a separate shell, X-server enabled (argument -X):

```
ros2 run image_tools showimage
```

For the cam2image node, set history to keep_last and depth to 1. This can be done in two ways:

1. Add and modify the parameters in the cam2image.yaml inside the package config folder. This is now not needed, but can be useful in Assignment 1.1.3.
2. Include the parameters as *ROS arguments* in the run command as follows:
`--ros-args -p depth:=1 -p history:=keep_last`

Show that the system works.

Note: you might check the tips in Section 0.2.1 on structuring the ROS2 files, finalising this assignment, and producing the report.

Questions

1. Describe in your own words what the parameters `depth` and `history` do and how they might influence the responsiveness of the system. Are the given parameter values good choices for usage in a sequence controller? Motivate your answer.
2. What is the frame rate of the generated messages? How can you influence the frame rate? Is there a maximum? What determines the maximum?

1.1.2 Adding a brightness node

Create a node that determines the average brightness of the image and, using some threshold, sends on a new topic if a the light is turned on (it is light) or off (it is dark).

Show that the system works (checking Section 0.2.1 might be beneficial).

1.1.3 Adding ROS2 parameters

Change the node in such a way that the threshold is a ROS2 parameter. Show that it is settable from the command line when starting the node (`ros2 run ... --ros-args -p ...`) as well as during run time (`ros2 param set ...`).

Show that the system works (checking Section 0.2.1 might be beneficial).

1.1.4 Simple object-position indicator

A very simple way of detecting the position of a coloured object (or bright light) in an image is the following:

1. Gray-scale the image.
2. Apply a threshold on the specific colour or brightness of each pixel.
If the threshold is well-chosen (and there are no other parts with the same colour or brightness in the image), the result is a black image with a large white dot where the coloured object or bright thing is. You may need to reduce background light.
3. Compute the ‘centre of gravity’ (CoG) of the white pixels. This CoG (in pixel coordinates) gives an indication of the position of the centre of the object.

Create a node that, given a camera input, outputs the position of the coloured object or bright light (if there is any) in pixel coordinates. The node should be easy to use, e.g., also when using in a larger project, on a different webcam or with different lighting conditions.

Show that the system works (checking Section 0.2.1 might be beneficial). Also, discuss your design choices.

Note: In Assignment 3, you have to follow a small *green* ball with the RELbot, using its front camera. With this subassignment, you could prepare for that.

1.2 Assignment 1.2: Sequence controller

Assignment 3 is on controlling a differential drive robot, called the *RELbot* (Figure 1.1). This robot is driven by two electromotors and can be considered as a rigid body that can move on a surface: driving forward or backward or turning. For describing the pose (position and orientation) of the robot, we only need x , y and θ_z , rotation around the vertical axis (z -axis), see Section 15.4 of the RELbot Manual (van Oort et al., 2025).

In order to prepare for the final assignment set, in this assignment we explore the ways of creating a sequence controller in ROS2 and control a simulation model of the RELbot.

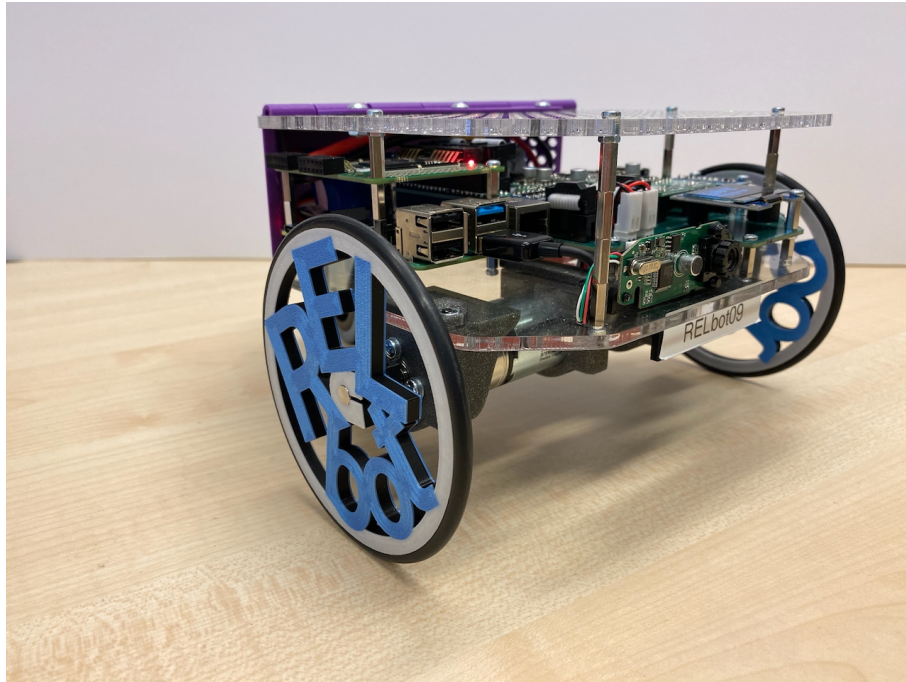


Figure 1.1: RELbot v1.1.

1.2.1 Unit test the sequence controller using the RELbot Simulator

A ROS2 node simulating the RELbot behaviour is available for this assignment (see Canvas). The node needs a webcam stream from a steady webcam (e.g., the webcam of your computer), this is achieved by using the previously used `cam2image` tool. Details about the use of the RELbot Simulator node is in Chapter 15 of the RELbot Manual (van Oort et al., 2025). On getting the webcam stream from your laptop to the ROS nodes, see Chapter 7 of the RELbot Manual.

Create a node that generates a sequence of setpoints and test it using the RELbot Simulator node.

Also create a launch file that sets up all required nodes.

Show that the system works (checking Section 0.2.1 might be beneficial) and discuss the design decisions you made.

Questions

1. Plot the setpoint and actual position (use of RQt is OK). Explain the results. Do not forget to look at the time constant of the first order system.

1.2.2 Integration of image processing and RELbot Simulator

This sub-assignment is an intermediate step towards the ultimate goal of having the RELbot Simulator use its own (moving) camera output.

In this sub-assignment the data from your webcam is used directly (software development goes small, testable, steps at a time).

Extend the sequence controller ROS2 node such that the RELbot Simulator follows a coloured object (or bright light) that you move around in front of your webcam. Keep the old functionality available, so adding a new method or object might be fruitful here. Use the steady (non-moving) stream of your laptop camera (not the `/moving_camera_output`) as an input for the object-position indicator node of Section 1.1.4 and compute appropriate setpoints from that.

Remember to keep the system modular (i.e., keep the node(s) easily re-usable; thus not too specialised).

Show that the system works (checking Section 0.2.1 might be beneficial) and discuss the design decisions you made.

Questions

1. Is the system you created an open-loop or closed-loop system (why)?
2. Is ROS2 (which is a non-realtime platform) in general, appropriate for these types of systems? Why? Are there any limitations (i.e., when do things go wrong)?

1.2.3 Classical only: Closed loop control of the RELbot Simulator

Classical Only

This sub-assignment is *not* for PPD students. Only for those who take the *Classical* version of ASDfR.

In this part you make RELbot look at the coloured object or bright light again, but this time the input of the object-position indicator should be the `/moving_camera_output` of the RELbot simulator (just as in the real case, where the camera mounted on RELbot moves around during operation).

In order to make this work, you need to steer the RELbot pose (position and orientation), $\mathbf{x}_{\text{RELbot}} = [x_{\text{RELbot}} \ \theta_{z,\text{RELbot}}]^T$, towards the (absolute) position of the light $\mathbf{x}_{\text{light}}$ such that the error, $\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}$, becomes zero.

A convenient controller choice is a first order controller, where the rate of change of the RELbot orientation, $\dot{\mathbf{x}}_{\text{RELbot}}$, is proportional to the error, i.e.,

$$\dot{\mathbf{x}}_{\text{RELbot}} = \begin{bmatrix} \dot{x}_{\text{RELbot}} \\ \dot{\theta}_{z,\text{RELbot}} \end{bmatrix} = \frac{1}{\tau} \cdot (\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}) = \frac{1}{\tau} \cdot \begin{bmatrix} x_{\text{light}} - x_{\text{RELbot}} \\ \theta_{z,\text{light}} - \theta_{z,\text{RELbot}} \end{bmatrix}; \quad \mathbf{x}_{\text{RELbot}} = \int \dot{\mathbf{x}}_{\text{RELbot}} dt, \quad (1.1)$$

where τ is some time constant (a good start would be $\tau = 1\text{s}$).

Notes

- The position of the coloured spot / bright light in the camera image is precisely $\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}$.
- Moving in the x direction means zooming in or zooming out.
- Moving around the z axis (changing θ_z) means horizontally moving the moving-camera cut out over the webcam image.
- The only way to influence the camera orientation is by sending a setpoint $\mathbf{x}_{\text{set}}$ to the simulator node. Fortunately, the RELbot can track its setpoint rather well, so we can assume that $\mathbf{x}_{\text{RELbot}} = \mathbf{x}_{\text{set}}$, resulting in

$$\dot{\mathbf{x}}_{\text{set}} = \begin{bmatrix} \dot{x}_{\text{set}} \\ \dot{\theta}_{z,\text{set}} \end{bmatrix} = \frac{1}{\tau} \cdot (\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}) = \frac{1}{\tau} \cdot \begin{bmatrix} x_{\text{light}} - x_{\text{RELbot}} \\ \theta_{z,\text{light}} - \theta_{z,\text{RELbot}} \end{bmatrix}; \quad \mathbf{x}_{\text{set}} = \int \dot{\mathbf{x}}_{\text{set}} dt. \quad (1.2)$$

Develop the system described above. For the integration (second equation of (1.2)) you can use the Forward Euler integration method.

Show that it works (i.e., that the simulated RELbot can track the object). Also discuss your design choices. Checking Section 0.2.1 might still be beneficial.

Questions

1. Is the system you created an open-loop or closed-loop system (why)?

2. Is ROS2 (which is a non-realtime platform) in general, appropriate for this type of systems? Why? Are there any limitations (i.e., when do things go wrong)?

1.3 Note – for both PPD and Classical versions of the course

Keep the nodes you have programmed for this assignment set, as you need them again in Assignment 3.

2 Assignment 2.0: Timing in Robotic / Cyber-Physical Systems

2.0 Introduction

In this assignment, you will analyse the timing of a Robotic or Cyber-Physical System, in which the SRT parts are ROS2 nodes. More specifically, it is about the timing of threads in the SRT and in the FRT part of the Raspberry Pi. By doing so, you can find out what SRT and FRT imply with respect to timing of the loops. In particular, what jitter appears in control loops. Factors such as computational load and how busy a processor obviously affects how good the timed loop adheres to the specified timing, and thus require investigation.

Furthermore, important is characterising roundtrip times: a bidirectional, synchronised communication between two nodes, like in closed-loop control systems. And of course, how such a roundtrip behaves under several load conditions.

Next to this, by characterising the Raspberry Pi with respect to timing, using a kind of bare implementation (that is, without the ROS2 stuff), you are also setting up a basic timer experiment on the Raspberry Pi.

As preparation for Assignment 3, the last part of the current assignment (Assignment 2.4) is on constructing diagrams of the ECS architecture of the RELbot system.

2.0.1 Raspberry Pi / RELbot in the Lab

The larger part of this assignment set consists of doing timing measurements on a Raspberry Pi. They are situated in the RaM Lab (Carré 3434). We have 6 Raspberry Pis in the form of RELbots available at the same time.

For this assignment, you do *not* need to use the same Raspberry Pi / RELbot during all time slots. The Raspberry Pis are identical, and you should have a copy of your files on your local computer anyway, so switching to a different Raspberry Pi should not be a problem.

Each group is allowed to use 2 two-lecture-hour time slots per week on a Raspberry Pi / RELbot, during the weeks this assignment is scheduled. Timeslots are available on Canvas for claiming.

You *must* subscribe to a time slot in Canvas before you are allowed to use it.

See Chapter 8 of the RELbot Manual (van Oort et al., 2025) for how to get access to the Raspberry Pis / RELbots in LabOne of RaM, CR 3437.

The time you have available at a Raspberry Pi is limited. Therefore, it is vital to be well-prepared:

- The programs of Assignments 2.1 and 2.3 (Classical only) can be fully prepared and tested at home on your Virtual Machine. If you program correctly, the code can run both on the VM and on the Raspberry Pi without any code modifications. Note that you *must* recompile though, throw away all object files and executables before recompilation, to force the build system to run a complete compile process again.
- The program of Assignment 2.2 can *not* be fully tested on the VM because your VM has no Xenomai functions. However, you can do a best-effort to write the program without compiling and testing the final version. VS Code's intellisense / autocorrect already helps to prevent typos.
- Most EVL / Xenomai 4 commands have different names than the comparable POSIX commands, such that compiling at 'home' (uh... on your VM) might not help that much. In case the difference is only function parameters, you can keep them during develop-

ment at the POSIX settings and only change them to the EVL / Xenomai 4 version when you work on the Raspberry Pi.

- Do any post-processing and report-writing later. However, you could do some initial (sanity) check of the results in the lab, to prevent leaving the lab with useless data.

2.0.2 Documentation and References

- Documentation for all POSIX C functions is conveniently embedded in Linux; you can use the command-line command `man` for this, e.g., from the Linux command line:
`man pthread_create`
Alternatively, an online version of the man-pages is at <https://man7.org/linux/man-pages/index.html>.
- Specifically for documentation on `CLOCK_REALTIME` and `CLOCK_MONOTONIC` (used in Section 2.1), refer to the manpage of `clock_gettime` (they are described in many other man pages as well, but in less detail).
- Documentation on Xenomai 4 is at <https://evlproject.org/overview/>.

2.1 Assignment 2.1: Timing of a periodic non-real-time thread (Ubuntu)

This subassignment is about timing tests on ‘normal’ Linux, whereas the next subassignment is about doing the *same* timing tests on our real-time Linux, so Xenomai.

Develop a program that spawns a POSIX thread which has a while (or for) loop in which the timed periodically-executed code runs. Use an OS timer or OS signals to *precisely* control the timing. The computation in the loop must be executed every 1.0 ms (timing of the loop is 1.0 ms, so mimicking a loop-control loop (e.g. PID control law) of 1 ms sample time).

Let the loop do some computational work so that it actually spends some time inside the loop.

Run the program twice: without and with extra processor load (from another program).

Investigate the timing of the loop. Discuss any particularities. Does extra computational load significantly affect the timing performance?

Notes and Tips

- The timing measurements should be taken when the program runs on the Raspberry Pi 4. However, you can prepare and test the program at home. See Section 2.0.1.
- Use `clock_gettime` for accurate time measurement.
- Carefully select and tune the timer or clock you choose to time the loop, as you need some precision in the timing.
- Avoid printing to screen/file IO inside the loop as this may heavily influence the timing of the loop.
- Make sure that the program exits nicely; e.g., the main thread waits for the thread to end, files and other resources are properly closed etc.
- For computational work, consider doing some math. Experiment a little with the amount needed to get a nice processor load (do not flood it though).
- You can use the command line tool `htop` to view the processor load.
- To generate extra processor load, you can use the command line tool `stress` (both on the virtual machine and the Raspberry Pi). For this, open an extra terminal or extra SSH connection and start `stress` in there before running your program. Think of suitable parameters and motivate them.

Questions

1. Explain what timing aspects we are interested in and why. Keep in mind that the course is on software development *for robotics*; a typical use case here is doing closed-loop control

- with a given sample time. Give at least two ways of representing the measured data. Discuss advantages and disadvantages of them.
2. Explain why you have chosen the way of timing the loop you have used. What are advantages and disadvantages of your method compared to other timer-realisation methods? *Hint:* think about clock functions and OS signals such as `sigwait`.
 3. What is the difference between `CLOCK_MONOTONIC` and `CLOCK_REALTIME`? Which one is better for timing a firm real-time loop?
 4. When using `timer_create/sigwait`, what happens with the timer you initialised when the program ends? And what happens when it crashes due to some error?
 5. Is it a good idea to use the approach used in this subassignment for closed-loop control of a robotic system? Why (not)?

2.2 Assignment 2.2: Timing of periodic real-time thread (Xenomai)

Make a variant of the program of Assignment 2.1 in which the periodic loop runs real-time under Xenomai.

Investigate the timing of the loop, that is, display the jitter in a diagram. Discuss any particularities. Does extra computational load significantly affect the timing performance?

Notes and Tips

- This program must run on Xenomai on the Raspberry Pi 4. Obviously, you can *not* fully prepare and test this at home on your VM. See Section 2.0.1.
- Most POSIX functionality can stay the same; however for EVL / Xenomai 4 names of those functions are different.
- Take care to make the program real-time capable.
- See Chapter 11 of the RELbot Manual (van Oort et al., 2025) for details on how to compile for EVL / Xenomai 4.
- You can still use `htop` to show processor load. However, this command only shows load from processes running on Linux, not from those running on EVL / Xenomai 4. To show the load by EVL / Xenomai 4 processes, use the command `evl ps -l` (switch is lowercase L). Apart from the load (%CPU), this command shows some other statistics, such as the number of *mode switches* (ISW) made.
- It is important to be aware of which CPU cores are reserved for Xenomai tasks on the Raspberry Pi. By default, Linux attempts to schedule processes on all available cores. However, you can isolate specific cores for Xenomai by adjusting the kernel boot parameters. The Raspberry Pi is configured such that Linux only schedules processes on cores 0, 2 and 3, leaving core 1 reserved for Xenomai. Even with this setting, you must still explicitly assign a process or thread to a EVL / Xenomai 4 core by setting its thread affinity.
- Factors such as memory allocation, system calls can easily disrupt real-time behaviour.
- Real-time programming requires careful selecting which constructs to use in the real-time part, as quite some constructs consume more runtime than expected, and thus contribute to jitter or add to delay of responses in the system.

Questions

1. For each POSIX / EVL / Xenomai 4 command used in your program, explain *in your own words* what it does and why you need it for a real-time program. If the command has arguments that influence the real-time behaviour (e.g., the scheduling type `SCHED_X`, or the clock type `CLOCK_X`), discuss why the chosen option is the best for real-time performance.
2. Compare the timing performance of this program with the program from Section 2.1. What can you conclude on timing with respect to real-time capability?

3. Is it a good idea to use the approach used in this subassignment for closed-loop control of a robotic system? Why (not)?

2.3 *Classical only*: Assignment 2.3: Timing of two communicating ROS2 nodes

Classical Only

This sub-assignment is *not* for PPD students. Only for those who take the *Classical* version of ASDfR.

Create two ROS2 nodes, Seq<nn> and Loop<nn>, where nn is your group number. Let *one* node provide the timer running on 1 kHz. Let Seq<nn> send a message each sample time to Loop<nn>. Let Loop<nn> send a message back to Seq<nn> *in reply* to the message it got from Seq<nn>.

Run the program twice: without and with extra processor load. Investigate the timing, that is show jitter, round-trip time, and jitter of round-trip time graphically. Discuss any particularities. Does extra computational load significantly affect the timing performance?

Tips and Remarks

- Extra provisions might be needed to get the ROS2 nodes running due to the mutual dependency w.r.t. inputs (subscribers).
- Again, the timing measurements should be taken when the program runs on the Raspberry Pi 4. However, you can prepare and test the program at home. See Section 2.0.1.
- Take into account the hints in Section 2.1.
- To match which sent and received messages belong together in Seq<nn>, it is wise to send unique messages (e.g., with unique reference numbers).
- When testing the round-trip time of messages, take into account that the policies which are set for the communication between ROS2 nodes affect the results. An example is enabling or disabling message buffering. Recall from Section 1.1.1 that you can affect these policies with `ros2 run ... --ros-args -p ...` See [Concepts/QoS Settings](#) for a list of arguments.
- Usually, a ROS2-node executes indefinitely and is stopped by completely stopping the program containing the node with Ctrl+C. However, when doing measurements, you probably want to run the experiment for a known, finite amount of time and do some post-processing in the same program afterwards. One way of accomplishing this, is to stop the node from within, which is done by calling `rclcpp::shutdown()` inside its functions. Doing this is not mandatory though.

Questions

1. Compare the results from this subassignment with the results from Assignment 2.1. Discuss the results.
2. Motivate your tuning of communication policies, that is, which parameters you have used, or not.
3. Discuss the analogy between the node architecture used in this subassignment and a typical robot control application. Put in your answer also the reasoning in what node you have placed the timer.
4. Is it a good idea to use the approach used in this subassignment for closed-loop control of a robotic system? Why (not)?

2.4 Assignment 2.4: RELbot CPS Architecture Diagrams

In this subassignment, you will construct diagrams to think ahead for the realisation of the ECS software in Assignment 3. Doing so, gives you a flavour of working along approaches of *model-based software development*.

Next to ROS nodes for soft real-time software, you need also code running in firm real-time.

Details on the RELbot including its hardware are in Chapters 13 – 16 of the RELbot Manual (van Oort et al., 2025). Details on XRF2, the firm real-time framework used on the RELbot are in Chapters 9 and 10 of the RELbot Manual (van Oort et al., 2025).

Block diagram

Construct a high-level block diagram of the structure of the Cyber-Physical System for which you are developing the embedded software. Cover the complete CPS / Robotic system.

- Detailing should be to the level of ROS 2 nodes and topics or the control stack as regularly shown on the lecture slides; similar detail is expected for the other parts. So, distinguish essential functionality and draw that as separate items (blocks).
- Clearly indicate the boundaries between non/soft/firm/hard real-time of the Cyber part.

Details of each block

Per item and communication channel, specify at least the following aspects:

- For each item:
 - Name.
The name must clearly indicate the function of the item; you may add a concise description separately from the diagram.
 - Environment (e.g., ROS2, Non-RT, Xenomai, mechanical).
 - Sample time (if applicable).
- For each communication channel:
 - Signal Name.
 - Type of channel (e.g., ROS topic, XDDP, IDDP, SPI, electrical).
 - Data type and size for vectors / arrays.
 - For signals representing physical quantities: unit (deg, rad, ...).

To avoid cluttered diagrams, or too small text sizes in the diagrams, you can use colour coding or abbreviations (adhere to conventions and standards). While doing so, do provide a legend or a list explaining the abbreviations.

Diagrams drawn on paper are accepted as long as they are readable and complete. A few suggestions for making diagrams on a computer:

- Draw.io (has both an online version on draw.io and an offline version (Help|Get Desktop in the online version)).
- UMLet (<https://www.umlet.com/>), specific for UML diagrams.
- Your own favorite application.

Both suggested tools have integration into VS Code; check the VS Code Extensions list.

FRT code parts to 'fill' the FSM

Specify what computation must be executed in the *user-programmable states* of the FSM of XRF2. Write it as pseudo code, and clearly indicate to which state of the FSM it belongs.

Specify which commands must be sent over the command channel (XenoCmd) to let the computation parts run.

The XRF2 FSM is shown in Figure 9.5 of the RELbot Manual and on Slide 3-52 (Slide 52 of Lecture 3). The XRF2 implementation overview is shown in Figure 9.2 of the RELbot Manual and on Slide 3-54.

Questions

1. Why are the states needed as you specified here? And why are the states you did *not* specify here *not* needed?
2. Assume the RELbot has a basic robot arm with basic gripper on its top, like in Figure 2.1. What states of the FSM would need computations? Indicate what functionality these (extra) computations must have.
3. Assume distance sensors are mounted next to the number plate for detecting obstacles (for instance Time-of-Flight sensors). In what part in the CPS should these be placed? In what block of the control stack should the sensor signal go to, and what function should it feed?



Figure 2.1: Idea of a simple robot arm that could be placed on top of the RELbot (Waveshare, figure is adapted).

3 Assignment 3 — Integration, Controlling a robot

3.0 Introduction

In this final assignment, you will control the real RELbot, version 1.1 (see Figure 1.1), such that it follows a coloured object. For this, you combine the ROS 2 nodes you made in Assignment 1 with a firm real-time controller part.

All code runs on the Raspberry Pi of the RELbot, the SRT parts on Linux (Ubuntu 22.04) with ROS2 *Humble* and the FRT parts run on EVL / Xenomai 4.

Notes

- We use on the RELbot indeed Ubuntu 22.04 and ROS2 *Humble*, so *not* the newest Long-Time Stable version. This due to issues concerning interfacing to the icoBoard containing the FPGA over SPI bus, which we could not resolve before starting this assignment. This is the reason why all ROS2 development for the RELbot uses ROS2 Humble on Ubuntu 22.04.
- The webcam is mounted upside down, because of placement constraints while keeping the camera sensor in the middle. This implies that you have to flip the image. This is supported in our *updated* cam2image ROS2 package, using an openCV function.

3.0.1 RELbot in the Lab

The larger part of this assignment consists of developing code for and on the RELbot's Raspberry Pi. Six RELbots are available in the RaM Lab (Carré 3434), the same location as for Assignment 2.

For this assignment, you do *not* need to use the same Raspberry Pi / RELbot during all time slots. The RELbots are identical (except a really small variation in some physical parameters) and you should have a copy of you files on your local computer anyway, so switching to a different Raspberry Pi should not be a problem.

Each group is allowed to use 2 two-lecture-hour time slots per week on a Raspberry Pi / RELbot, during the weeks this assignment is scheduled. Timeslots are available on Canvas for claiming.

You *must* subscribe to a time slot in Canvas before you are allowed to use it.

See the RELbot Manual (van Oort et al., 2025), Chapter 8 for access to the RELbots and its Raspberry Pi; Chapters 13 – 16 for details of the RELbot and its software; and Chapters 9 – 11 for the XRF2 framework, its logger, and how to compile for Xenomai 4.

The time you have at a setup is limited, therefore it is vital to be well-prepared. Just like for Assignment 2.

Structure your ROS2 projects as specified in Chapter 6 of the RELbot Manual and adhere to the submission instructions in the same chapter. Just like for Assignment 1.

3.0.2 Planning and intermediate feedback

For each subassignment you have the opportunity to let a TA check your work. During that check, you must demonstrate to the TA that the subassignment works. The TA can then provide you with tips. Use this check as a dedicated feedback moment.

To support you keeping on track, a suggested planning for this assignment set is given in Table 3.1.

Note: Week 14 and 15 are shorter, due to the UT being closed during the Eastern weekend.

Week	Assignment	Topic
13	3.1	Assignment-1 code on RPi of RELbot
14*	3.2 + 3.3	Test XRF2 + LoopController
15*	3.4	Full integration
16	3.5	Slack time; Extras

Table 3.1: Suggested planning.

3.1 Assignment 3.1: Controller-code stack on the Raspberry Pi of the RELbot

Reproduce the experiment from Assignment 1 on the Raspberry Pi of the RELbot, without activating the motors. So *only* using the Raspberry Pi and its webcam. That is, still keep using the RELbot simulator instead of the real motors.

This to only test the effect of using other computer hardware (Raspberry Pi and camera), and *not* the loop control and mechanical part, as that these parts are added in a later subassignment. In other words, is the Raspberry Pi processor strong enough to run the code, and computes the results in the same way as on the laptop (a Raspberry Pi is less powerful compared to a laptop)?

Show that the system works (checking Section 0.2.1 might be beneficial).

Tips and Remarks

- Re-use the nodes you developed in Assignment 1.
Do improve your nodes based on the generic and specific feedback you received. For students in the PPD groups: we provide a ROS2 node with the functionality of the solution of Assignment 1.2.3.
- Green stress balls are available to serve as the coloured object.
- The RELbot USB camera is connected to the Raspberry Pi.
- The webcam on RELbot v1.1 is mounted upside down due to placement constraints. So, you need to rotate the image 180° first. This is provided in the Cam2Image nodes that are available on Canvas.
- RQt is *not* installed on the RELbot's Raspberry Pi (it *cannot* easily be installed). Fortunately, *showimage* is available.
- To show graphical results, you can use *X forwarding*, the same as during ROS 2 development using your VM, see Chapter 4 of the RELbot Manual. The Raspberry Pi on the RELbot does not have a graphical environment, so it is quite obvious that you cannot see contents of graphical windows directly on the Raspberry Pi.
- You can use the command line tool *htop* to view the processor load.
- **Note:** Store results of this subassignment separately, that is, prepare it for submission (see Chapter 6 of the RELbot Manual) and set it apart such that it can be graded separately. Obviously, this implies that for the ROS2 code, you make a separate ROS2 package per subassignment.

Questions

1. Does the system perform as well as on your own laptop? Why? What is the processor load?
2. Did you have to make changes to the *experiment* to get it running on the Raspberry Pi? If so, why was it necessary? Could you have avoided it if you had done the experiment in Assignment 1.2.3 differently?

3. Did you have to make changes in the *nodes* you developed in Assignment 1.2.3? If so, why was it necessary? Could you have avoided it if you had developed the nodes in Assignment 1.2.3 differently?

Sign off and results

You can show the result of Assignment 3.1 to one of the TAs in order to receive feedback. You can use this as a dedicated feedback moment.

Describe your tests / experiments and add relevant test data to the report (checking Section 0.2.1 might be beneficial).

3.2 Assignment 3.2: XRF2: the FRT software architecture on RELbot's Raspberry Pi

In this assignment, you install and check / test XRF2, the architecture (software structure) of the Firm Real-Time software for the RELbot. So, you are running XRF2 *without* adding the loop-controller code and other functionality. This assignment is also about using the Xenomai 4 – ROS 2 bridge and framework (XRF2), and test it as-is separately.

Set up XRF2 and connect that to both the ROS 2 side and the RELbot hardware, but *without* any controller functionality. So pass the signals from the SRT part directly to the HRT part and vice versa. In fact, you are creating a test bed for the architecture of this part of the system.

Construct a separate test bed to test your FRT part with its connections, whereby:

- The ROS 2 node *only* generates signals to feed to the FRT part. So, this node is the unit-test node.
- The FRT program passes values to its neighbours (ROS 2 node and RELbot HRT part and hardware)
- the RELbot is on its stand, to prevent moving away. You do drive the motors, so the wheels turn!!

Show that the system works (checking Section 0.2.1 might be beneficial) and discuss the design decisions you made. Record / store the results of your tests to add to your report of this assignment.

Remarks and Tips

- *Keep the robot on its stand!!*
This to prevent unintendedly running away of the RELbot, which can cause serious damage.
- Make a separate ROS 2 node with essential functionality for testing purposes only. You could use ROS parameters to select different input patterns, to have a variety of tests easily available.
- One of the ways to produce the architecture for the loop controller is to create an 'empty' loop controller in 20-sim and generate code from that, and use that in the framework. Indeed, the focus is on testing *only* the use of the XRF2 framework.
- You may *only* extend the parts of XRF2 as indicated in Chapter 9 of the RELbot Manual. You would violate the hand-in rules otherwise (See Chapter 6 the RELbot Manual) and also destroy modularity. If you need extra functionality, use the places where it is allowed to extend / add code. If you think you absolutely need to update code elsewhere, discuss with the teaching assistant first.
- Be careful what diagnostic / debug statements you use in the Xenomai 4 part, as stepping out the *out-of-band* situation is quite easy, and often happens unintendedly. You could use `evl_print()` to show results / numbers in a Xenomai 4 terminal window.
- To test the signal path of the encoder signals, you can carefully rotate the wheels by hand.

- To test the signal path of the PWM signals, you can measure the result of the wheel movement by some external means.
- Be sure to have the motors turn positively (from the point of view of steering value) when the robot has to move forward (camera faces in the forward direction). The same holds for the encoders: these should give pulses such that an increasing value of the angle is produced when the RELbot 'moves' forward.
- **Note:** Store results of this subassignment separately, that is, prepare it for submission (see Chapter 6 of the RELbot Manual) and set it apart such that it can be graded separately. Obviously, this implies that for the ROS2 code, you make a separate ROS2 package.

Questions

1. What are advantages and disadvantages of separately testing the FRT *skeleton* part first, as done in this assignment?
2. What is the advantage of starting with an 'empty' loop controller in 20-sim and use that as a starter for Assignment 3.3?

Sign off and results

You can show the result of Assignment 3.2 to one of the TAs in order to receive feedback. You can use this as a dedicated feedback moment.

Describe your tests / experiments and add relevant test data to the report (checking Section 0.2.1 might be beneficial).

3.3 Assignment 3.3: Loop-Controller algorithm in FRT part

Complete the Firm Real-Time software for the RELbot, by putting the loop-controller algorithm in the FRT software architecture of the previous assignment (Assignment 3.2). The loop controller is available in the 20-sim model of the RELbot, see Chapter 16 of the RELbot Manual.

Generate the loop-controller code from the 20-sim model and put that in your FRT software framework. Again, Chapter 16 of the RELbot Manual provides details how to fill the *control loop* with the loop-controller code, and to let it run in the *run* state of the FRT part.

Functionality of the Measurement & Actuation (M&A) block must be put in the same part of the FRT software framework. Best is to test that in 20-sim and generate code from there together with generating the code for the loop controller. Otherwise, you would need to update the code generated from 20-sim.

Motivate the choice of what you put in these parts, especially on scaling factors.

You can *reuse* and extend the testbed of Assignment 3.2. You could even do a few tests that are the same as in the previous assignment.

Show that the system works (checking Section 0.2.1 might be beneficial) and validate the correctness of controller as final test by letting the RELbot ride in a straight line, and let it make a 90° corner.

Remarks and Tips

- *Keep the robot on its stand, except for the final test!!*
This to prevent unintendedly running away of the RELbot, which can cause serious damage.
- Chapter 16 of the RELbot Manual describes a 20-sim model with a dynamic model of the RELbot and an appropriate loop-controller.
- Keep in mind the block diagram you made in Assignment 2.4.
- Make sure the directions (i.e., which direction is called positive) adhere to those mentioned in Section 15.4 of the RELbot Manual.

- Do first some basic tests, with the RELbot on its stand.
- *Let the RELbot ride only after you have checked its loop-controller functionality thoroughly while keeping the robot on its stand.*
- **!!! Check first with the TA before letting the robot ride on the table or floor !!!**
- You may only extend the parts of the XRF2 as indicated in the RELbot Manual (Chapter 9). You would destroy modularity otherwise. If you need extra functionality, use the places where it is allowed to extend / add code. If you think you absolutely need to update code elsewhere, discuss with the teaching assistant first.
- If you do not want to use 20-sim, you can use any other modelling package (e.g., Simulink) for code generation. In that case, you need to design the position controller yourself, in such a way that it fits to the provided interface. XRF2 has a subclass for this situation, see Figure 9.6 of the RELbot Manual.
- **Note:** You *must* do code-generation from some modelling package; writing the controller directly in C/C++ is not accepted.
- **Note:** Store results of this subassignment separately, that is, prepare it for submission (see Chapter 6 of the RELbot Manual) and set it apart such that it can be graded separately. Obviously, this implies that for the ROS2 code, you make a separate ROS2 package or node.

Questions

1. What are the advantages and disadvantages of doing code generation (as opposed to writing the controllers directly in C/C++)?
2. Why is it beneficial to generate *all* code needed in the FRT parts from 20-sim (that is, avoid the need of adding extra conversions etc – often denoted as *glue code*)?

Sign off and results

You can show the results of Assignment 3.3 to one of the TAs in order to receive feedback. You can use this as a dedicated feedback moment.

Describe your tests / experiments and add relevant test data to the report (checking Section 0.2.1 might be beneficial).

3.4 Assignment 3.4: Final integration

Finally, integration time!!

Create the full system of the *RELbot following a coloured object* by composing all blocks you have built and individually tested in the previous sections.

The goal is to let the code work immediately right ("*Doing right first time*"), to optimally profit of all the hard work in the previous steps, namely carefully developing and testing the different parts that are combined in this step.

Show that the final system works (checking Section 0.2.1 might be beneficial); discuss your design choices.

Record one or more videos to make plausible that your RELbot functions as intended. In case your videos are really large w.r.t. file size, a link to some shared space is appreciated.

Draw a block diagram of the architecture / structure of the total system (CPS), such that it describes the results obtained in this last assignment.

Note: This block diagram must be a representation of the real situation. Obviously, it is *not* identical to the answer of Assignment 2.4.

Remarks and Tips

- *Keep the robot on its stand, especially during the first tests of the integrated system!!*
This to prevent unintendedly running away of the RELbot, which can cause serious damage.
- You might need to tune the controllers such that the RELbot reacts rather slow, to prevent unintendedly riding too fast. After some first tests you could let the RELbot ride faster. However, this would not be necessary.
- Generate some measurements to underpin that the RELbot really does what you describe in the report about its functioning.
- **Note:** Store results of this subassignment separately, that is, prepare it for submission (see Chapter 6 of the RELbot Manual) and set it apart such that it can be graded separately. Obviously, this implies that for the ROS2 code, you make a separate ROS2 package.

Questions

1. Compare the resulting system block diagram with the block diagram from Assignment 2.4. Indicate the differences, and describe why the final implementation deviated from the original diagram. (Note: mismatch between simulation and reality is often the case, so seeing differences is not at all a fault from your side!).

Sign off and results

You can show the results of Assignment 3.4 to one of the TAs in order to receive feedback. You can use this as a dedicated feedback moment.

Describe your tests / experiments and add relevant test data to the report (checking Section 0.2.1 might be beneficial).

3.5 Assignment 3.5: Extra functionality (Bonus assignment)

In this bonus assignment you can add functionality to the already working setup. Some ideas:

- Start-up procedure
- Safety layer: RELbot stops after seeing an obstacle.
- Advanced behaviour in the case no coloured / green object is seen.
- Extra safety: stop the movement of the whole robot if one of the degrees of freedom does not function as expected.
- Your own ideas...

Describe what extra functionality you want to make and how it will influence the behaviour of the whole system. Explain how you implemented the extra functionality. Motivate the design choices. Show and discuss test results.

Remarks and Tips

- As with all assignments, the focus is on developing *good, well-structured solutions*. A very cool feature which is just 'hacked' into the system will not give you points.
- **Note:** Store results of this subassignment separately, that is, prepare it for submission (see Appendix F of the Software-Tools Manual) and set it apart such that it can be graded separately. Obviously, this implies that for the ROS2 code, you make a separate ROS2 package or nodes.

Sign off and results

You can show the results of Assignment 3.5 to one of the TAs in order to receive feedback. You can use this as a dedicated feedback moment.

Describe your tests / experiments and add relevant test data in the report (checking Section 0.2.1 might be beneficial).

3.6 Final Submission of Assignment 3

For the final submission, obviously combine all results of the subassignments of this Assignment Set. Keep in mind the following:

- Adhere to Chapter 6 of the RELbot Manual for providing ROS 2 results of the subassignments. Hand in *only* ROS2 code you made yourself, so *never* add ROS2 code we provided.
- For each subassignment in which you produce Xenomai 4 software, submit a structure as indicated in Chapter 9 of the RELbot Manual. Indeed, the untouched files appear several times in your submission. Just accept this inefficiency.
- Provide readme files containing the build and run commands, allowing for easy checking your software.
- Of course, adhere to Section 0.2.1 w.r.t. what and how to submit.

A Changelog

- **Version 2025.1, 4 February 2026**

Initial version, consisting of:

- First version of the Assignment Manual.

B Course-specific framing for PPD projects

As stated in the CBL manual, the course-specific framing is:

- SDfR
Describe the CPS architecture of the robotic system.
- ASDfR
Describe the CPS architecture of the robotic system. Include also the real-time aspects.

Further details are presented here. First the common things, then specifics for ASDfR, and also in case your group has both SDfR and ASDfR students on board.

B.1 Describe the CPS architecture of your robotic system – both SDfR and ASDfR

It is on presenting the architecture of the software, interfacing to hardware, and hardware of your robotic system / embedded control system / cyber-physical system.

From the point of view of developing software for robots, the terms embedded control system and cyber-physical system are often used interchangeably.

Provide at least the following

- Block diagram of the ‘top level’ of the system (high-level block diagram), comparable to ROS2 nodes, and use a similar detail for non-ROS2 parts.
- Next to this block diagram, specify of each software part, next to its name in the diagram:
 - Description of its function.
 - Where it runs, that is on which processor (type), if applicable.
 - Sample time (if applicable).
 - Interfaces do not need to be specified but may be mentioned, as details of the connected communication channels also indicate what is produced as output, expected / received as input.
- Specify for each communication ‘channel’, next to its name in the diagram:
 - Type of channel (ROS, data, network package, electrical signal).
 - Data type, size of data.
 - Physical quantity and unit, in case physical signals are communicated.
 - Sample time (if applicable).
- Specify for the non-software parts:
 - Description of its function.
 - What domain it is in (electrical, mechanical...).
 - If available and useful, more domain-specific details.
 - Interfaces do not need to be specified, unless the communication channels are not described separately (for instance user interface, interface to environment). Then describe that interface.

B.2 Specifics for ASDfR

In addition to the information stated for SDfR, add:

- Augment the block diagram to indicate timing regime: Non / Soft / Firm-Hard real-time / Physics / etc. You may indicate those timing regimes by drawing (dotted) lines to separate those areas.
- What timing support each software block needs to have: SRT, F/HRT, or NonRT.
- What QoS and timing behaviour each communication channel needs to have. You may use ROS2 terminology, if applicable and useful.
- How connections between SRT and F/HRT parts are realised.

B.3 Combined SDfR and ASDfR

In case your CBL group has both SDfR and ASDfR students, you may specify both parts as a combination.

B.4 Hints on producing diagrams

Diagrams drawn on paper are accepted as long as they are readable and complete. A few suggestions for making diagrams on a computer:

- Draw.io (has both an online version on draw.io and an offline version (Help|Get Desktop in the online version)).
- UMLet (<https://www.umlet.com/>), specific for UML diagrams.
- Your own favorite application.

Both suggested tools have integration into VS Code; check the VS Code Extensions list.

Bibliography

Student Charter (2020). *Student Charter at University of Twente*. URL: <https://www.utwente.nl/en/ces/sacc/regulations/charter>.

van Oort, G., L.B.L. Lenders, and Broenink (2025). *RELbot Software and Usage Manual (v2025.n)*. Manual.